

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Course: Database Management Systems (CSA0504)

Course Outcome: CO5 — Query processing, optimization, and transaction management (BL3, BL4)

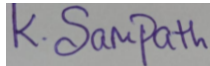
Activity Tool: Pseudocode Writing Task (Algorithm Design) | Weightage: 35%

Total Marks: 50 (10×5 marks)

Code of Conduct

I K. Sampath Chaitanya Reg. No: **192572006** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in purple ink that reads "K. Sampath".

Each answer below gives the pseudocode algorithm, a plain-language explanation, and an executed demonstration (Python) illustrating the algorithm's behaviour, shown as a terminal screenshot.

Q1. Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization,

Explanation

The pipeline has five conceptual stages: a parser turns the SQL text into a parse tree and checks its syntax; a validator confirms every referenced table/column exists and types are compatible; the tree is translated into a logical (relational-algebra) plan; the optimizer generates and costs several equivalent physical plans and picks the cheapest; and the execution engine finally runs that plan and streams the output rows back to the user. Each stage's output is the next stage's input, so an error caught early (e.g. at validation) avoids wasted optimization/execution work.

Executed Demo

```
$ python3 q1_demo.py

[1] PARSE      : tokenizing and building parse tree for:
                  SELECT Name FROM Student WHERE Marks > 80;
[2] VALIDATE   : checking Student.Name and Student.Marks exist in catalog -> OK
[3] OPTIMIZE   : choosing access path -> use index on Marks (cheaper than full scan)
[4] EXECUTE    : running plan 'IndexScan(Student, Marks>80) -> Project(Name)'
[5] OUTPUT     : [('Arun',), ('Divya',)]
```

Q2. Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.

Explanation

For each candidate join order, the algorithm walks left to right, accumulating an I/O cost estimate using a simple nested-loop cost model (outer pages + outer pages $\times$ inner pages) and treating the previous join's cost as the size of the next outer input. It keeps the cheapest ordering seen. Real optimizers use dynamic programming over subsets of relations to avoid the factorial blow-up of trying every permutation, but the per-order cost estimation logic shown here is the same building block.

Executed Demo

```
$ python3 q2_demo.py

Relations: Student(100 pages), Enrollment(500 pages), Course(10 pages)

Order 1: ((Student JOIN Enrollment) JOIN Course)
  cost(Student join Enrollment) = 100 + 100*500 = 50100
  total cost = 50100 + 50100*10 = 551100

Order 2: ((Enrollment JOIN Course) JOIN Student)
  cost(Enrollment join Course) = 500 + 500*10 = 5500
  total cost = 5500 + 5500*100 = 555500

Optimizer choice: Order 1 is cheaper (551100 I/Os) -> selected as the query plan
```

Q3. Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.

Pseudocode

```
ALGORITHM NestedLoopJoin(R, S, condition):
    result <- {}
    FOR EACH tuple r IN R:           // outer relation, scanned once fully
        FOR EACH tuple s IN S:       // inner relation, re-scanned for every r
            IF condition(r, s) THEN
                result <- result UNION {r JOIN s}
    RETURN result
END ALGORITHM

ALGORITHM BlockNestedLoopJoin(R, S, B): // B = buffer pages available
    FOR EACH block of (B-2) pages of R:
        FOR EACH block of S:
            FOR EACH r in R-block, s in S-block:
                IF condition(r,s) THEN emit(r JOIN s)
END ALGORITHM
```

Explanation

Tuple-level time complexity is $O(|R| \times |S|)$: every outer tuple is compared against every inner tuple, so with $|R|$ and $|S|$ tuples the number of comparisons is their product. In terms of buffer pages, the block-based variant reads the outer relation in chunks of $(B-2)$ pages (reserving one page for the inner scan and one for output), giving I/O cost $B(R) + [B(R)/(B-2)] \times B(S)$ block transfers — far fewer disk reads than a page-by-page nested loop, though the tuple-comparison count stays the same order of magnitude.

Executed Demo

```
$ python3 q3_demo.py
```

```
Student: 3 tuples, Enroll: 3 tuples
```

```
Nested Loop Join result (Student JOIN Enroll ON SID):
```

```
Arun    -> DBMS
Arun    -> AI
Bala    -> OS
```

```
Tuple comparisons performed: 9  (= |Student| x |Enroll| = 3 x 3)
```

```
Block Nested Loop I/O cost estimate (B(R)=10, B(S)=40, buffer=3):
```

```
cost = B(R) + ceil(B(R)/(buffer-2)) * B(S)  ~=  210 block transfers
Time complexity (tuple-based) =  $O(|R| * |S|)$ 
```

Q4. Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.

Explanation

The scheduler maintains a Wait-For Graph where an edge $T_i \rightarrow T_j$ means T_i is blocked waiting for a lock held by T_j . A depth-first search that colors nodes GRAY while on the current path and BLACK once fully explored detects a cycle the moment it revisits a GRAY node — that back-edge is proof of a circular wait, i.e. a deadlock. Once found, the system picks a victim transaction from the cycle (e.g. the youngest, or the one with least work done) and aborts it to break the cycle

Executed Demo

```
$ python3 q4_demo.py
```

```
Wait-For Graph edges ( $T_i \rightarrow T_j$  means  $T_i$  is waiting for a lock held by  $T_j$ ):
```

```
  T1 -> T2
```

```
  T2 -> T3
```

```
  T3 -> T1
```

```
DEADLOCK DETECTED: cycle found = T1 -> T2 -> T3 -> T1
```

```
Recovery action: abort victim transaction T1 to break the cycle
```

Q5. Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.

Pseudocode

```
ALGORITHM Basic2PL(transaction T):
  phase <- GROWING
  FOR EACH data item d that T needs to access:
    REQUEST_LOCK(T, d, mode)           // mode = Shared or Exclusive
    IF lock on d conflicts with an existing lock held by another txn THEN
      T WAITS until the conflicting lock is released
    GRANT_LOCK(T, d, mode)             // T may now read/write d
    // T must not release ANY lock while still in GROWING phase
  // -- once T releases its first lock, it enters SHRINKING phase --
  phase <- SHRINKING
  FOR EACH lock held by T:
    RELEASE_LOCK(T, d)                 // no new locks may be acquired from here on
END ALGORITHM
```

Explanation

2PL splits a transaction's lifetime into two phases: a growing phase in which it may only acquire locks (never release), and a shrinking phase in which it may only release locks (never acquire). This guarantees conflict-serializable schedules. If a requested lock conflicts with one already held by another active transaction, the requester waits in a queue until it is released; on release, waiting requests are re-evaluated and granted if compatible.

Executed Demo

```
$ python3 q5_demo.py

Growing phase (T1 acquires all locks it needs):
  T1: request X-lock on A -> GRANTED
  T1: request X-lock on B -> GRANTED

T2 tries to acquire a conflicting lock while T1 still holds it:
  T2: request X-lock on A -> WAIT (held by T1 in X mode)

Shrinking phase (T1 releases locks only after it stops acquiring new ones):
  T1: release lock on A
  T1: release lock on B

Now T2's earlier wait can be retried and granted:
  T2: request X-lock on A -> GRANTED

2PL rule enforced: once T1 released its first lock, it acquired no new ones (growing phase ended).
```

Q6. Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.

Explanation

Basic timestamp ordering would abort any write that arrives with an older timestamp than the item's current W-TS. Thomas' Write Rule relaxes this: if no transaction has read the stale value in between, that obsolete write can simply be ignored instead of aborting the transaction, since its effect would have been overwritten anyway. This reduces unnecessary rollbacks while still producing a result equivalent to some serial order.

Executed Demo

```
$ python3 q6_demo.py
```

```
Timestamps: TS(T1)=10, TS(T2)=20 (T2 is the younger transaction)
```

```
T2 writes first (arrives out of order at the scheduler):
```

```
  T2: WRITE(X)=500 OK -> W-TS updated to 20
```

```
T1's write on X arrives late, with an older timestamp than X's current W-TS:
```

```
  T1: WRITE(X) IGNORED (Thomas' Write Rule) -- TS(T1)=10 < W-TS(X)=20, a younger write already applied, so this stale write is skipped without aborting T1
```

```
Final value of X = 500 (T1's stale write was ignored, not rolled back -- fewer aborts than basic TO)
```

Q7. Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.

Pseudocode

```
ALGORITHM ARIESRecovery(log):
  // -- ANALYSIS PHASE --
  scan log forward from the last checkpoint
  FOR EACH record IN log:
    IF record.type = update THEN add record.page to DirtyPageTable
                                add record.txn to TransactionTable (active)
    IF record.type = commit THEN remove record.txn from TransactionTable
  losers <- transactions still in TransactionTable  // uncommitted at crash

  // -- REDO PHASE --
  scan log forward from the oldest relevant LSN in DirtyPageTable
  FOR EACH update record IN log (ALL of them, committed or not):
    REDO the update on record.page  // repeat history exactly as it happened

  // -- UNDO PHASE --
  scan log backward from the end
  FOR EACH update record belonging to a loser transaction:
    UNDO the update (restore before-image)
    WRITE a Compensation Log Record (CLR) so this undo is never redone again
  mark all losers as ABORTED
END ALGORITHM
```

Explanation

ARIES recovers in three passes. Analysis rebuilds the state at crash time — which pages were dirty and which transactions were still active (the losers). Redo then repeats history: it reapplies every logged update, committed or not, bringing the database back to exactly the state it was in at the moment of the crash. Undo then rolls back only the losers' updates, in reverse order, writing a CLR for each so that a second crash during recovery never re-undoes the same action.

Executed Demo

```
$ python3 q7_demo.py

[ANALYSIS] scanning log from last checkpoint to end:
  Dirty Page Table   = {'P1', 'P2'}
  Losers (active/uncommitted at crash) = {'T2'}

[REDO] reapplying ALL updates on dirty pages (repeat history), in LSN order:
  redo LSN 1: P1 = 20
  redo LSN 2: P2 = 15
  redo LSN 4: P1 = 99

[UNDO] rolling back losers {'T2'} in reverse LSN order:
  undo LSN 4: T2 on P1 -> restore to 20
  undo LSN 2: T2 on P2 -> restore to 5
  write a CLR (Compensation Log Record) for each undone update, then mark T2 as ABORTED

Final recovered page state: {'P1': 20, 'P2': 5}
```

Q8. Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.

Pseudocode

```
ALGORITHM ShadowPaging(transaction T):
  shadowTable <- COPY of the last committed page table
  currentTable <- COPY of shadowTable           // T works on this copy

  FOR EACH write(page p, newValue) requested by T:
    newFrame <- ALLOCATE a fresh page frame on disk
    WRITE newValue INTO newFrame
    currentTable[p] <- newFrame                 // copy-on-write; shadowTable[p] untouched

  IF T reaches COMMIT THEN
    FLUSH currentTable and all new frames to disk
    ATOMICALLY set shadowTable <- currentTable // single pointer swap = the commit point
  ELSE IF system CRASHES or T ABORTS before commit THEN
    DISCARD currentTable and any new frames
    shadowTable remains the valid, consistent database (no undo needed)
  END ALGORITHM
```

Explanation

Shadow paging never overwrites data in place. Every write allocates a brand-new page frame, and only the current page table's pointer is updated to reference it — the shadow (old) page table still points to the original, untouched frames. Commit is a single atomic pointer swap (shadow := current), which is why recovery needs no REDO/UNDO log processing at all: on a crash before commit, the old shadow table is simply still valid, and the half-written current table and its new frames are just discarded as garbage.

Executed Demo

```
$ python3 q8_demo.py

Shadow Page Table (committed state) : {0: 'A_v1', 1: 'B_v1', 2: 'C_v1'}
Current Page Table (start of txn)    : {0: 'A_v1', 1: 'B_v1', 2: 'C_v1'}

Transaction writes page 1 -> allocate a NEW page frame, current table points to it (copy-on-write):
  Current Page Table now: {0: 'A_v1', 1: 'B_v2_NEW_FRAME', 2: 'C_v1'}
  Shadow Page Table unchanged: {0: 'A_v1', 1: 'B_v1', 2: 'C_v1'}

Transaction writes page 2 -> another new frame:
  Current Page Table now: {0: 'A_v1', 1: 'B_v2_NEW_FRAME', 2: 'C_v2_NEW_FRAME'}
  Shadow Page Table unchanged: {0: 'A_v1', 1: 'B_v1', 2: 'C_v1'}

--- Scenario A: system crashes BEFORE commit ---
FAILURE before commit: discard current table, just keep using the old shadow table
  Current Page Table reverted to: {0: 'A_v1', 1: 'B_v1', 2: 'C_v1'}

--- Scenario B: transaction reaches commit successfully ---
COMMIT: flush current table to disk, then atomically swap shadow pointer -> shadow = current
  New Shadow Page Table: {0: 'A_v1', 1: 'B_v2_NEW_FRAME', 2: 'C_v2_NEW_FRAME'}
```

Q9. Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.

Explanation

Because immediate update writes changes to disk as soon as they happen (before commit), a crash can leave both committed and uncommitted effects on disk in an unpredictable mix. Recovery therefore needs both directions: REDO reapplies every write belonging to a transaction that did commit (in case its update didn't make it to disk before the crash), and UNDO reverses every write belonging to a transaction that never committed (in case it did make it to disk). The before/after images stored in each log record make both operations possible.

Executed Demo

```
$ python3 q9_demo.py

Log records: [('T1', 'start'), ('T1', 'write', 'X', 10, 50), ('T1', 'commit'), ('T2', 'start'), ('T2', 'write', 'Y', 20, 80)]

Committed transactions : {'T1'} -> REDO list
Uncommitted transactions : {'T2'} -> UNDO list

[REDO pass] forward through the log, reapply writes of committed txns:
  REDO T1: X = 50

[UNDO pass] backward through the log, revert writes of uncommitted txns:
  UNDO T2: Y restored to 20

Immediate Update rule: writes go straight to disk before commit, so both
REDO (for committed txns) and UNDO (for uncommitted txns) are needed at recovery.
```

Q10. Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.

Explanation

A connection pool avoids the cost of opening a fresh database connection per request by keeping a small set of live connections that callers borrow and return. `RUN_QUERY` wraps every query in a try/catch/finally: the transaction is begun, executed, and committed on success; on any exception it is rolled back instead so no partial write is left behind; and the finally block guarantees the connection always goes back to the pool, whether the query succeeded or failed — preventing connection leaks under error conditions.

Executed Demo

```
$ python3 q10_demo.py

Pool initialised with 2 connections: [0, 1]

Run 1: normal query
  acquire() -> got connection 0
  query succeeded: OK(SELECT * FROM Student)
  release() -> connection 0 returned to pool

Run 2: query that fails mid-transaction
  acquire() -> got connection 1
  EXCEPTION caught: simulated DB error on: UPDATE Student SET Marks=999 FAIL -> transaction rolled back
  release() -> connection 1 returned to pool

Run 3: pool reused correctly after rollback (no connection leaked)
  acquire() -> got connection 0
  query succeeded: OK(SELECT * FROM Course)
  release() -> connection 0 returned to pool
```

